

JAVA

OBJECT-ORIENTED PROGRAMMING

Warehouse Inventory Tracker

Design and Implementation using Object-Oriented Programming in Java



Student Name

[Enter your name]



College Name

[Enter your college]

InventoryManager Class & Key Features

This section outlines the core methods of our `InventoryManager` class and highlights its essential features for efficient stock control.

Core Methods



addProduct()

Integrate new items into the inventory system.



updateQuantity()

Adjust stock levels in real-time to reflect changes.



searchProduct()

Locate specific products quickly using various criteria.



viewProducts()

Access a comprehensive display of all stored products.



deleteProduct()

Permanently remove products no longer in stock.



showLowStockProducts()

Identify items nearing critical reorder points.

Key Features Highlighted



Product Addition

Seamlessly expand your product catalog.



Product Search

Quick access to specific inventory details.



Low Stock Alerts

Proactive warnings for critical items.



Real-time Updates

Accurate stock data, always current.

Validation & Sample Output

Ensuring data integrity and providing a clear overview of inventory status.

Input Validations

- Quantity cannot be negative.
- Product name cannot be empty.
- Threshold must be a valid positive number.
- Product not found error handling for queries.
- Prevention of duplicate product IDs during entry.

Sample Output: Inventory Table

Product ID	Product Name	Quantity	Threshold	Price	Status
SKU001	Laptop Pro X	50	20	\$1200.00	In Stock
SKU002	Ergonomic Keyboard	10	5	\$75.00	Low Stock
SKU003	Wireless Mouse	2	10	\$25.00	Out of Stock
SKU004	4K Monitor 27"	75	30	\$350.00	In Stock
SKU005	HD Webcam	0	5	\$40.00	Out of Stock

Advantages & Future Enhancements

Advantages of Current System

-  Easy and intuitive inventory management
-  Faster product tracking and retrieval
-  User-friendly console interface
-  Reduces manual work and errors
-  Real-time stock monitoring

Future Enhancements

-  Database integration (MySQL/PostgreSQL)
-  GUI application (Swing/JavaFX)
-  Web-based version (Spring Boot)
-  Barcode scanning capability
-  User authentication and roles
-  Mobile app integration

Product Class Structure

Defining the blueprint for inventory items with essential attributes and behaviors.

1

Class Variables

- `productId`: Unique identifier for each product.
- `productName`: Descriptive name of the product.
- `quantity`: Current stock level in the warehouse.
- `threshold`: Minimum stock level to trigger a "low stock" alert.
- `price`: Unit price of the product.

2

Key Methods

- `getters` and `setters`: Provide controlled access to product data, ensuring encapsulation.
- `toString()`: Returns a user-friendly string representation of the product object for display and logging.
- **Stock Status Logic**: Internal methods to check if the product is **in stock**, **low stock** (below threshold), or **out of stock** (quantity is zero).

Technologies & OOP Concepts

Technologies



Java Programming
Language



ArrayList for dynamic
storage



Scanner Class for user
input



Console-Based
Application

OOP Concepts



Classes and Objects



Encapsulation (private
variables)



Constructors for
initialization



Method Overriding and
polymorphism

Project Objectives



Efficient Product Management

To efficiently manage the inflow, storage, and outflow of warehouse products, optimizing space and accessibility.



Minimize Manual Errors

To significantly reduce human errors in inventory tracking and record-keeping through automated processes.



Real-time Stock Monitoring

To enable continuous, up-to-date monitoring of all stock levels within the warehouse environment.



Low Stock Alerts

To implement an alert system that automatically notifies relevant personnel when product stock falls below predefined thresholds.



Effective OOP Application

To effectively apply core Object-Oriented Programming (OOP) concepts in Java for a robust and scalable solution.

System Architecture



This flowchart outlines the structured flow of data and control within the Inventory Tracker, demonstrating how each class and component contributes to the system's overall functionality.



Introduction to Inventory Management

Effective inventory management is the backbone of efficient operations, ensuring that businesses maintain optimal stock levels to meet demand without incurring excessive costs or waste.

Why Warehouse Tracking Systems Matter

Prevent Stockouts

Ensure products are always available to meet customer demand, avoiding lost sales and dissatisfaction.

Reduce Waste & Costs

Minimize overstocking, spoilage, and obsolescence, leading to significant savings and improved profitability.

Boost Efficiency

Streamline warehouse operations, from receiving to dispatch, for quicker turnaround times and optimized workflows.

The Imperative for Automation



Eliminate Manual Errors

Automated systems drastically reduce human error in data entry and tracking, ensuring accuracy.



Save Time & Resources

Tasks that once took hours can be completed in minutes, freeing up staff for more strategic activities.



Enhance Data Accuracy

Real-time updates and precise tracking lead to highly reliable inventory data for better decision-making.

Conclusion

- Successfully implemented the Warehouse Inventory Tracker system.
- Improved inventory handling and management processes through practical application.
- Effectively applied Java OOP concepts, including classes, encapsulation, and methods.
- Demonstrated a strong understanding and practical application of core programming concepts.



Thank You